INTRODUZIONE

Una delle esigenze più comuni del WEB è quella di restringere l'accessibilità di una o più pagine di un applicazione web ai soli utenti autenticati o addirittura ad una ristretta cerchia di utenti (ad es. agli utenti admin). Questa esigenza si presenta anche nel mondo delle API. Nella prima parte di questo articolo spiegherò il principio su cui si basa JWT mentre nella seconda parte scriveremo una semplice implementazione di autenticazione Spring su JWT.

AUTENTICAZIONE STATELESS

Un servizio REST dovrebbe essere per definizione stateless: ovvero le interazioni tra client e server devono essere senza stato, rispettando la comunicazione stateless nativa del protocollo HTTP. La gestione dello stato (se necessaria) avviene sul client, ottimizzando le prestazioni del server che non deve curare lo stato. L'approccio stateless di un architettura RESTful rende quindi inappropriato l'utilizzo dei cookie di sessione per il meccanismo di autenticazione.

La soluzione di autenticazione Stateless piu' semplice è l'autenticazione BASIC: ad ogni chiamata del client viene passato un token in un header http contenente le credenziali in modo che il server ad ogni chiamata possa autenticare il client. In questo modo mantengo stateless il mio backend RESTful ma ottengo un overhead dovuto al fatto che il server debba autenticare ad ogni richiesta il client. Questa soluzione non è quindi adatta a sistemi di produzione: immaginiamo di dover ri-eseguire ad ogni richiesta le query di autenticazione sul nostro db su un sistema con migliaia di utenti online nel singolo istante! Una soluzione completamente Stateless, scalabile e con nessuno overhead è quella oggetto di questo articolo: JWT (JSON WEB TOKEN)

JSON WEB TOKEN

Prima di analizzare il meccanismo di autenticazione su JWT cerchiamo innanzitutto di capire cos' è un JSON WEB TOKEN: abbreviato più comunemente con l acronimo di JWT. JWT è uno standard che viene usato per "regolare" le richieste tra due parti. JSON Web Token è un security token (una stringa) che agisce come un container per le claims degli user. I claims sono informazioni di un utente: chi è l'utente loggato (e altre info di contorno: ad es mail, nome, cognome), scadenza di un token (quindi

scadenza dell'autenticazione dell'utente), privilegi dell'utente (è un admin o un normale utente) ed altre info customizzabili a seconda del contesto applicativo . Tali informazioni sono firmate lato server secondo la specifica JSON Web Signature (JWS), quindi il server è in grado di riconoscere se sono state generate da lui o meno. Riportiamo un esempio di token JWT:

eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhZG1pbiIsImF1ZGllbmNlIjoid2ViIiwicm9sZXMiOlsiUk9MRV9BRE1JTiIsIlJPTEVfVVNFUiJdLCJpc0VuYWJsZWQiOnRydWUsImV4cCI6MTUxMjk3NTQ1MCwiaWF0IjoxNTEyOTY4MjUwMjMwfQ.J49qG_yi0yMZP4K2PWddTNR7QyUloAR7qj_3QtLL_6A

Come si evince dall'esempio identifichiamo 3 parti in un token JWT

- Header : Se decodifichiamo la prima parte del token scopriamo che si riferisce ad un json che contiene le due voci "typ" e "alg". Il primo ha come valore sempre "JWT", mentre il nodo "alg" contiene il nome dell'algoritmo usato per il token. Nell'immagine sopra è stato usato l'algoritmo "HMAC-SHA256".

- Payload : La seconda parte del token è il corpo vero e proprio che contiene dei dati "variabili" in base al contesto in formato json e codificati in base64. Nel nostro esempio il payload decodificato è


```
{
 "sub": "admin",
 "aud": "web",
 "roles": [
   "ROLE_ADMIN",
   "ROLE_USER"
```

```
 ],
 "isEnabled": true,
 "exp": 1512975450,
 "iat": 1512968250230
}
```


JWT propone dei campi “standard” da utilizzare nel payload (es: sub, audience, exp e iat) ma il programmatore può inserire proprietà aggiuntive (vedi isEnabled e roles da me definiti e non contemplati nei clams standard). Andando ad analizzare il json del body possiamo interpretare dal token le seguenti info:

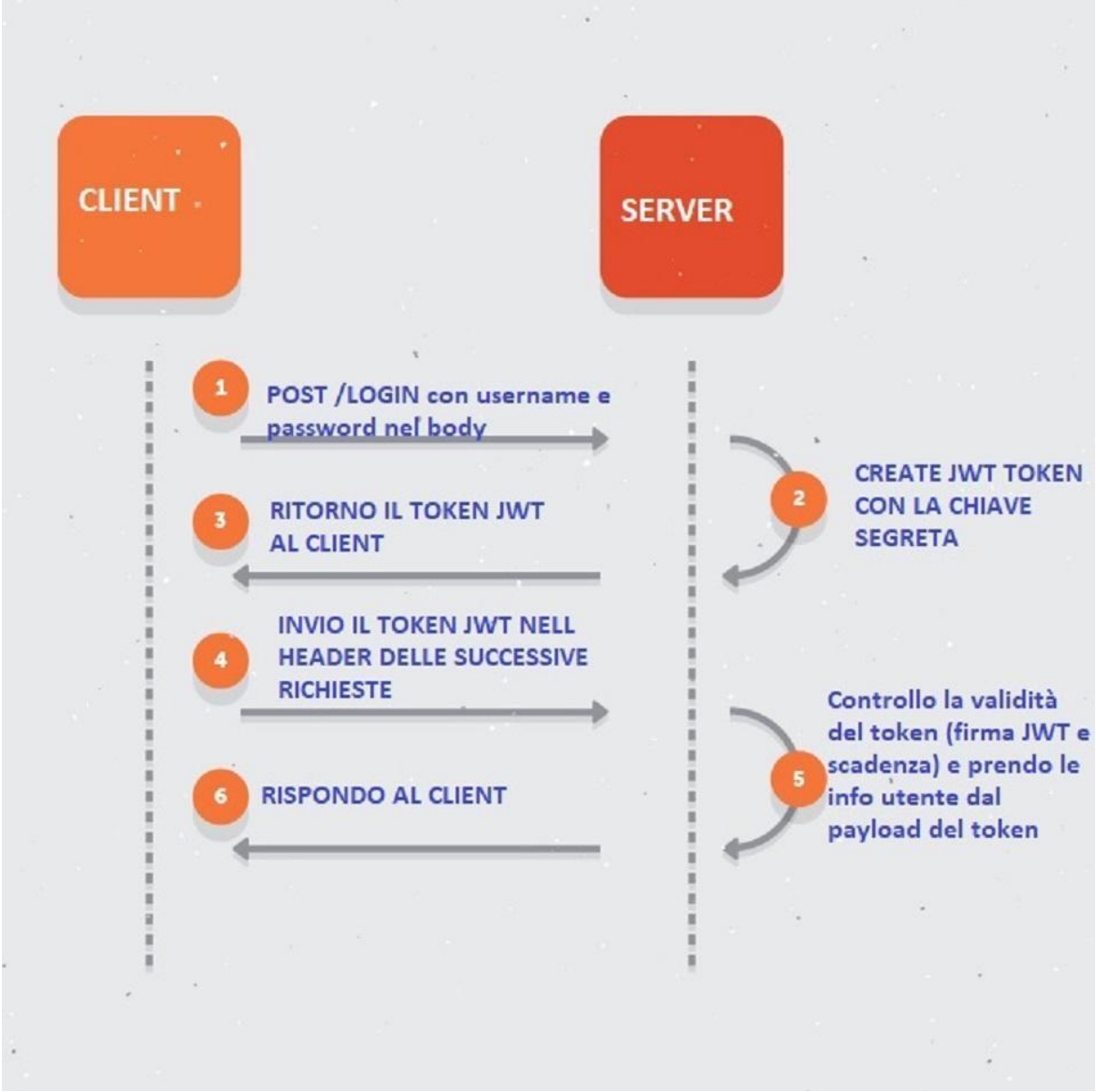
- utente admin (claim sub)
- ha come target il web (claim aud)
- ruoli utente ROLE_ADMIN, ROLE_USER (claim roles)
- utente abilitato (claim isEnabled)
- info relative al momento in cui è stato creato il token (iat) e sulla data di scadenza (exp): momento in cui tale token non sarà più ritenuto valido dal server.

- Signature: La terza parte di un token è la firma che non è altro che il risultato di una funzione hash 256 che prende in input la codifica base64 dell’header concatenandola con un punto alla codifica base64 del payload, il tutto codificato con la nostra “chiave segreta” che solo il server conosce.

Con questa piccola introduzione sui JSON WEB TOKEN capiamo subito che questi token possono esser un ottimo modo per trasportare all’interno di ogni richiesta HTTP info legate all’autenticazione di un utente in modo totalmente sicuro: il server grazie al campo signature è in grado di capire che tali info sono state generate da lui grazie alla firma e che quindi garantiscono che tale richiesta è da considerarsi relativa ad un utente autentificato.

FLUSSO LOGICO DI AUTENTICAZIONE JWT

Ora che abbiamo introdotto i JSON WEB TOKEN descriverò come possono essere utilizzati per implementare il meccanismo di autenticazione in un backend RESTful. Il meccanismo di autenticazione JWT può essere riassunto in questo schema.



1. Il client, utilizzando l'endpoint di login, invia le proprie credenziali al server per autenticarsi.

2. Il server verifica se le credenziali postate sono corrette. In caso positivo costruisce un token jwt utilizzando la chiave segreta e inserendo nel payload del token le informazioni utente e le data di creazione e scadenza del token; Tale token verrà firmato e autentificato con la sua chiave privata (importante solo il server deve conoscere la chiave privata)
3. tale token viene inviato nella risposta all'interno di un header http (nella implementazione che proporremo sotto utilizzeremo l header http X-auth)
4. Il client salva il token jwt risultato dell'autenticazione, e nelle successive richieste richieste inoltra sempre nell'apposito header http X-auth il token restituito nel punto 3. Tale token funge da "passaporto" per garantire che tali successive richieste sono autenticate.
5. Il server riceve il token ad ogni successiva richiesta. Ogni volta si assicura che il token sia stato firmato e autentificato con la sua chiave privata ed infine che non sia scaduto. Poiché il token contiene al suo interno il payload con le informazioni necessarie all'autenticazione (es. username e ruoli utente), il server eviterà di fare query ogni volta per verificare a quale utente corrisponde quel token e di recuperare le info utente necessarie per ricostruire lo stato della sessione (ottimo risultato per la scalabilità ed eliminando totalmente l'overhead della soluzione BASIC descritta ad inizio articolo).
6. Se passati i controlli descritti dal punto 5, il server tratterà l'utente come autenticato e gestirà la richiesta, altrimenti risponderà alla richiesta con un codice di errore HTTP 401 (utente non autorizzato).

INTEGRAZIONE CON AUTHENTICATION PROVIDERS

Quando ci si integra con un **Authentication Provider esterno** tramite standard moderni come **OAuth 2.0** e **OpenID Connect (OIDC)**, il flusso si evolve in questo modo:

1. **Spostamento delle credenziali**: Il proprio server Express non riceve né memorizza le password in chiaro o cifrate. La responsabilità di verificare "chi è l'utente" viene interamente delegata al provider esterno.
2. **Firme Asimmetriche (RS256 vs HS256)**: Poiché il token viene generato dal provider esterno e consumato dalla nostra API, non si usa più una chiave segreta simmetrica condivisa. Il provider firma il JWT utilizzando la propria **chiave privata** (algoritmo asimmetrico, solitamente RS256). La nostra applicazione Express scarica periodicamente le **chiavi pubbliche** del

provider da un endpoint standardizzato denominato **JWKS** (*JSON Web Key Sets*) per verificare l'integrità e l'autenticità della firma.

3. **Preservazione dell'architettura Stateless**: Una volta che l'applicazione Express ha convalidato la firma del token tramite la chiave pubblica del provider, il funzionamento rimane strettamente *stateless*. Il server estrae i dettagli dell'utente (i *claims*, come ID, email o ruoli) direttamente dal payload del token , senza la necessità di interrogare il database del provider o memorizzare sessioni sul server. Tutto il traffico deve comunque essere protetto da **HTTPS** per evitare attacchi Man-in-the-Middle.